TestHuset

Python Handout

FUNDAMENTAL DEVELOPMENT

1 Running Python

```
python --version    # is Python installed?
python hello.py     # run a file
python              # interactive, exit() to quit
```

In VS Code: open the folder, create a `.py` file, and run it with the ▶ button or from the terminal (`Ctrl + ``).

Tip: on some machines the command is `python3` or `py`.

2 Print and comments

```
print("Hello, world!")
print("Age:", 30)    # several values

# A comment explains why. Python ignores it.
```

3 Variables and data types

```
name = "Alex"       # str
age = 30            # int
price = 19.95       # float
is_member = True    # bool

print(type(age))    # <class 'int'>
```

Names use `snake_case`, start with a letter, and are case sensitive. A variable can be given a new value at any time.

4 Input and converting types

`input()` always returns text. Convert before you calculate.

```
name = input("Your name: ")
age = int(input("Your age: "))
bill = float(input("Bill amount: "))

text = str(age)     # number back to text
```

5 Arithmetic

Operator	Meaning	Example	Result
`+ - *`	add, subtract, multiply	`2 + 3 * 4`	14
`/`	division (always float)	`7 / 2`	3.5
`//`	whole division	`7 // 2`	3
`%`	remainder	`7 % 2`	1
`**`	power	`2 ** 3`	8

```
total = total + 5
total += 5              # same thing, shorter
round(12.3456, 2)       # 12.35
```

6 Strings

```
name = "Alex"
greeting = "Hi " + name          # join text
message = f"{name} is {age}"     # f-string
price = f"{19.5:.2f}"            # '19.50'

len(name)        # 4
name.upper()     # 'ALEX'
name.lower()     # 'alex'
name[0]          # 'A', counting starts at 0
```

7 Indentation

Python uses indentation to show which lines belong together. Use 4 spaces. A line ending in `:` starts a new block.

```
if age >= 18:
    print("Adult")       # inside the if
print("Done")            # always runs
```

8 Comparisons and logic

Operator	Meaning
`==` `!=`	equal to, not equal to
`<` `>`	less than, greater than
`<=` `>=`	less or equal, greater or equal
`and`	both sides must be true
`or`	at least one side must be true
`not`	flips true and false

Watch out: `=` stores a value, `==` compares two values.

9 if, elif, else

```
if age < 3:
    price = 0
elif age < 12:
    price = 50
else:
    price = 100

if age >= 12 and is_member:
    price = price - 20
```

Python checks from the top and runs the first branch that is true. Conditions can be nested inside each other.

10 while loops

Repeat as long as the condition is true.

```
count = 1
while count <= 3:
    print(count)
    count += 1     # or it never stops
```

```
break        # leave the loop now
continue     # skip to the next round
```

Stuck in a loop? Press `Ctrl + C` in the terminal.

11 Lists, for and range()

```
animals = ["lion", "zebra", "owl"]
animals[0]               # 'lion'
animals.append("seal")   # add to the end
len(animals)             # 4

for animal in animals:
    print(animal)

for i in range(5):       # 0 1 2 3 4
    print(i)
for i in range(1, 6):    # 1 2 3 4 5
    print(i)
```

12 Accumulator pattern

Start with an empty result and build it up one round at a time.

```
prices = [50, 100, 100]
total = 0
for price in prices:
    total += price
print(total)             # 250
```

13 Functions

```
def greet(name):                 # parameter
    print(f"Hello, {name}!")

greet("Alex")                    # argument

def add_tip(bill, percent=15):   # default value
    return bill * (1 + percent / 100)

total = add_tip(200)             # 230.0
total = add_tip(200, 10)         # 220.0
```

print or return? `print` shows a value on screen. `return` hands the value back so the rest of the program can use it.

14 Reading error messages

Read the last line first, then the line number above it.

Error	Usually means
`SyntaxError`	Missing `:`, quote or bracket
`IndentationError`	Spaces do not line up
`NameError`	Misspelled or undefined name
`TypeError`	Mixing types, e.g. text + number
`ValueError`	Bad conversion, e.g. `int("abc")`
`IndexError`	List position does not exist